

Alignment and Integration Mandate: A Prescriptive Update for the Algorithmic Trading System Project Plan

Section 1: Executive Summary of Strategic Imperatives

This document provides a prescriptive and actionable guide for updating the "Features, Phases & Integration Strategy - Algorithmic Trading System" project plan. The modifications detailed herein are mandated by the strategic directives and architectural principles outlined in the "Strategic Analysis and Architectural Recommendations" document. The objective is to ensure the project's execution plan is fully aligned with the firm's evolved technological and business ambitions for its quantitative trading capabilities.

The core strategic pivot driving these changes is the transformation of the project's fundamental purpose. The original project plan described a tactical, single-asset, self-contained trading system designed for rapid deployment of a simple strategy. The new strategic vision, however, mandates the creation of an institutional-grade, multi-asset, and Machine Learning (ML)-driven trading platform. This platform is no longer viewed as a simple execution utility but as a core, alpha-generating asset for the firm, designed for extensibility, scalability, and long-term competitive advantage.

This shift in identity from a tactical tool to a strategic platform is profound. The original project was a contained engineering initiative with a limited scope. The revised project is now a cornerstone of the firm's future in quantitative research and data science. This elevation in strategic importance necessitates a fundamental re-evaluation of the project's scope, resourcing, and governance. The updated plan must reflect this new gravitas, accounting for the need for specialized personnel, including quantitative analysts, data scientists, and DevOps specialists, alongside significantly more rigorous senior management oversight.

The primary mandates from the strategic analysis that necessitate the comprehensive changes detailed in this report are as follows:

- **Architectural Paradigm Shift:** A transition from a monolithic application architecture to a distributed system of microservices communicating over an event-driven backbone is required. This is a non-negotiable architectural mandate intended to enhance scalability, resilience, and deployment agility.
- **Evolution of Signal Generation:** The system must evolve from executing simple, hard-coded, rules-based algorithms to supporting a sophisticated, Python-based Machine Learning framework. This includes the infrastructure to host, execute, and monitor complex quantitative models.
- **Expansion of Data Ingestion:** The platform's data capabilities must be expanded beyond a single market data feed to include a diverse range of sources, such as alternative data sets and cryptocurrency exchange feeds. This requires a formal data ingestion and normalization strategy.
- **Institutional-Grade Risk and Compliance:** The risk management framework must be upgraded from simple, locally configured rules to a robust, centralized, and compliant system. This includes real-time pre-trade checks, system-wide kill-switch functionality, and an immutable audit trail for regulatory compliance.
- **Stringent Non-Functional Requirements (NFRs):** The platform is now subject to high-frequency trading (HFT)-grade performance requirements, including targets for sub-millisecond latency and the ability to scale to handle a tenfold increase in data and order volume.

The collective impact of these mandates is substantial. They render the original project plan, timeline, and technical approach obsolete. The following sections provide a detailed, section-by-section set of instructions for modifying the project plan to align with this new strategic reality, ensuring the project is set up for successful execution against its new, more ambitious goals.

Section 2: Detailed Feature Set Modifications

The strategic shift towards a platform-based approach requires a fundamental restructuring of how system features are defined, categorized, and tracked. The original feature list, focused on user-facing functionality, is insufficient to capture the significant infrastructure and backend service development now required. This

section details the necessary additions, modifications, and deprecations to the project plan's feature set.

2.1 New Feature Additions: Platform Services

A critical flaw in applying the original project plan to the new architecture is the lack of visibility for foundational infrastructure components. Headless backend services such as a messaging backbone or a centralized risk engine represent significant design, development, and testing efforts. Treating them as implicit "enablers" for other features, rather than as first-class deliverables, leads to a gross underestimation of project scope and a critical project planning risk.

To rectify this, a new top-level feature category, titled "**Platform Services**," must be added to the project plan. This category will contain the core, often headless, microservices that form the backbone of the new architecture. This ensures that their development is explicitly planned, resourced, and tracked as a primary project deliverable. The following new Platform Services must be added:

- **PS-01: Event-Driven Messaging Backbone:** This service represents the implementation of Apache Kafka as the central nervous system for the entire platform, as mandated by the new architectural principles. The feature definition must include the setup and configuration of a high-availability Kafka cluster, the definition of key data topics (e.g., market-data.normalized, orders.new, executions.reports), the establishment of message schemas using a formal framework like Avro or Protocol Buffers for data governance, and the implementation of security and access control policies.
- **PS-02: Multi-Source Market Data Gateway:** This new microservice is responsible for all external market data connectivity. It will be designed to ingest data from diverse sources, including traditional equity feeds, cryptocurrency exchange APIs (e.g., WebSocket or REST), and alternative data providers. Its core responsibilities include data connection management, normalization of disparate data formats into a canonical internal model, and publishing the normalized data onto the Event-Driven Messaging Backbone for consumption by downstream services.
- **PS-03: ML Strategy Execution Service:** This service directly replaces the concept of a simple, monolithic strategy engine. It will be a dedicated, containerized service, likely implemented in Python to align with modern data science practices, designed to host and execute ML-based trading models. Its feature scope includes creating a secure environment for model execution, defining a standard API for loading models and feeding them market data, and providing robust monitoring and logging for model performance and behavior.
- **PS-04: Centralized Real-Time Risk Service:** This is a critical new component for ensuring institutional-grade control and compliance. This microservice will act as the sole gatekeeper for all trading activity. It will subscribe to order request events from the messaging backbone and perform a series of pre-trade risk checks against a centralized, database-driven limits repository. The feature definition must include the implementation of checks for order size, price deviation, and daily position limits, as well as the critical system-wide "kill-switch" functionality mandated by the new risk framework.
- **PS-05: Immutable Compliance Audit Service:** To meet stringent regulatory requirements (e.g., MiFID II), this service will be responsible for creating a complete, unalterable record of all trading activity. It will consume every order lifecycle event from the Kafka bus—from creation through to final execution or cancellation—and persist it to a compliant, immutable data store, such as a write-once-read-many (WORM) storage solution or a blockchain-based ledger.
- **PS-06: Quantitative Feature Store:** This new data repository is a foundational component for the ML-driven strategy. It is designed to store and serve pre-computed data features (e.g., rolling volatility, moving averages, sentiment scores) used for both training ML models offline and running them in live production. This service decouples feature engineering from model execution, enabling faster iteration by quantitative researchers and ensuring consistency between research and production environments.

2.2 Enhancements to Existing Features

Several features from the original plan remain relevant but require significant modification to function within

the new event-driven, microservices architecture.

- **F-12: Real-time Position Monitoring (Modification):**
 - **Original Concept:** This feature was described as a component that queries a local database on a polling interval, such as every 5 seconds, to update a user's view of their positions.
 - **Required Modification:** This feature must be completely re-architected. It will be re-implemented as a "read-side" microservice that operates asynchronously. Instead of polling a database, it will subscribe directly to the stream of execution reports on the Kafka backbone. It will build and maintain an in-memory, real-time state of all positions for immediate display. The 5-second polling interval is no longer acceptable; the requirement must be updated to "event-driven, sub-second updates" to align with the new high-performance NFRs. The service will persist periodic snapshots of its state for recovery purposes.
- **F-08: Order Execution Gateway (Replacement):**
 - **Original Concept:** This was described as a tightly coupled component responsible for connecting to a single broker, "Broker X," using the FIX 4.2 protocol.
 - **Required Modification:** This component must be rebuilt as a standalone, stateless microservice. Its sole responsibility will be to manage connectivity to external trading venues. It will listen for "new order" commands on a dedicated Kafka topic, translate these canonical order objects into the appropriate broker-specific format (e.g., FIX), and manage the full lifecycle of the order with the broker. Crucially, it must be designed with a pluggable architecture, where the core logic is separate from broker-specific "connectors." This ensures that adding support for new brokers in the future is a localized, low-risk change, not a system-wide overhaul.

2.3 Feature Deprecations and De-scoping

The new strategic direction renders certain concepts and features from the original plan obsolete. They must be explicitly removed from the project plan to avoid confusion and misdirected effort.

- **DEP-01: Monolithic Application Core:** The central concept of a single, monolithic application containing all business logic is fundamentally incompatible with the new strategy. This concept must be formally deprecated and replaced with documentation describing the distributed microservices architecture.
- **DEP-02: Local Configuration Files for Risk:** The original plan's approach of managing risk limits via local properties files is explicitly forbidden under the new institutional-grade risk mandate. This feature must be deleted and its requirements fully superseded by the Centralized Real-Time Risk Service (PS-04).

Section 3: Re-sequencing the Development Phases and Timeline

The shift from a simple monolith to a complex, distributed platform has profound implications for the project's execution roadmap. The original phasing, which prioritized the rapid deployment of a single trading strategy, is no longer viable or logical. The project's risk profile has inverted: the primary risk is no longer whether a specific strategy is profitable, but whether the complex underlying technology can be built reliably and performantly. Therefore, the development phases must be completely redefined to prioritize the mitigation of architectural and technical risk first.

3.1 Redefinition of Phase 1 (MVP): From Feature-First to Architecture-First

The original Minimum Viable Product (MVP) was focused on delivering a functional trading strategy in the shortest possible time. The new strategic direction mandates a different priority for Phase 1. The objective is no longer immediate business functionality but is instead to **retire architectural risk** by building and validating the foundational "scaffolding" of the new distributed system.

This redefinition of the MVP from a "Minimum Viable *Product*" to a "Minimum Viable *Platform*" is a critical shift that must be clearly communicated to all project stakeholders. Business sponsors, who typically associate

an MVP with a tangible, user-facing deliverable, must understand that the primary value of the new Phase 1 is technical, not commercial. Its goal is to prove that the core architectural choices are sound and that the platform can support future development. Failing to manage this expectation will lead to perceptions of slow progress. The project plan must be prefaced with a new introduction to the "Phases" section that explicitly re-frames the goal of Phase 1 as "Architectural Validation and Risk Reduction," establishing new criteria for success that are aligned with this technical objective.

The deliverables for the new, architecture-first Phase 1 are as follows:

1. **Production-Ready Messaging Backbone:** Deployment and operationalization of a high-availability Kafka cluster, which serves as the system's central nervous system.
2. **CI/CD and Observability Foundation:** Establishment of automated Continuous Integration/Continuous Deployment (CI/CD) pipelines for building, testing, and deploying containerized microservices. This includes foundational logging, metrics, and tracing infrastructure.
3. **Core Service Skeletons:** Development of skeleton implementations of the three most critical services: the Market Data Gateway, the Order Execution Gateway, and the Centralized Real-Time Risk Service. These initial versions will have minimal functionality but will prove that the services can be deployed, communicate via Kafka, and be managed by the CI/CD pipeline.
4. **End-to-End "Tracer Bullet" Implementation:** Implementation of a single, simple trading path through the new architecture. This involves:
 - Ingesting a single market data feed via the Market Data Gateway.
 - Running the original, simple "moving average" strategy inside the new containerized ML Strategy Execution Service (PS-03).
 - Sending the resulting order command to the Centralized Risk Service for a basic "allow-all" check.
 - Passing the approved order to the Order Execution Gateway for transmission to a test environment at the broker.
5. **Baseline Performance Validation:** A successful demonstration that this end-to-end flow functions correctly and can meet a baseline latency target, proving the viability of the core architecture.

3.2 Definition of Subsequent Phases

With the architectural foundation validated in Phase 1, subsequent phases can focus on building out functionality and integrating with the wider enterprise ecosystem.

- **Phase 2: Feature Enrichment and Enterprise Integration:** This phase will focus on adding depth to the core platform services. Key objectives include implementing the full suite of pre-trade checks in the Risk Service (PS-04), building out the Immutable Audit Service (PS-05), and integrating the platform with critical enterprise systems. This includes establishing the data pipeline to publish all trade and order lifecycle events to the firm's central Data Lake and integrating the Risk Service with the enterprise-wide Risk Management System (RMS) for more complex, portfolio-level checks. This phase will also see the onboarding of the first true ML-based model into the Strategy Execution Service.
- **Phase 3: Scalability, Performance, and Multi-Asset Expansion:** The focus of this phase shifts to hardening the platform for full-scale production use. This includes extensive performance engineering and tuning to meet the stringent NFRs for latency and throughput. It also involves onboarding additional brokers to the Order Gateway and expanding the Market Data Gateway to ingest new asset classes, such as cryptocurrencies and alternative data, as required by the strategic roadmap. Concurrently, the ML infrastructure will be enhanced with the development and integration of the Quantitative Feature Store (PS-06) to accelerate the research-to-production lifecycle for new models.

Section 4: Evolution of the Integration Strategy

The original project plan envisioned a self-contained system with minimal external touchpoints—essentially a single connection to one broker. The new strategic mandate transforms the system into a highly connected hub within the firm's enterprise architecture. This necessitates a complete replacement of the original integration

plan with a formal, robust strategy that addresses both external connectivity and deep internal integration.

4.1 External Connectivity: Brokers and Data Feeds

The previous approach of a hard-coded connection to "Broker X" is brittle and unscalable. It must be replaced with a strategy centered on a **"Pluggable Connectivity Layer."** This architectural principle dictates that the core services—specifically the Market Data Gateway (PS-02) and the Order Execution Gateway (F-08)—must be designed with standardized internal APIs for handling data and orders.

The actual communication with external parties will be handled by separate, isolated "connector" or "adapter" modules. For example, the Order Gateway will have a core logic module that understands the canonical internal order format and a separate "FIX Connector for Broker X" module that handles the specifics of the FIX protocol for that counterparty. This design ensures that adding a new data feed or a new broker becomes a matter of developing a new, self-contained connector module, a task that can be performed with minimal impact on the rest of the system. This approach is the practical implementation of the mandate to support multiple data sources and brokers.

4.2 Internal Enterprise Integration

This is a completely new section that must be added to the project plan, as the original "self-contained" model is now obsolete. The platform's success is now contingent on its ability to seamlessly integrate with key enterprise systems. This introduces a new class of project risk related to external dependencies, which were not present in the original plan. The success of the project is no longer solely in the hands of the development team; it is now critically dependent on the APIs, performance, release schedules, and support of the teams managing these enterprise systems.

To manage this risk, the integration strategy must be expanded to include a formal **Dependency Management Plan**. This plan must identify the specific teams and individuals responsible for the dependent systems, define formal Service Level Agreements (SLAs) for API performance and availability, and establish a cadence of regular cross-team synchronization meetings. This proactive approach is essential to mitigate the risk of project delays caused by these external dependencies.

The specific integration points that must be defined are:

- **Data Lake Integration:** The platform is a critical source of data for the firm. The integration pattern will be asynchronous and event-driven. The system will publish all final trade and order lifecycle events to a dedicated, well-defined Kafka topic. The firm's central Data Lake ingestion process will then consume from this topic, ensuring a clean decoupling of the systems. The Dependency Management Plan must secure a commitment from the Data Lake team on the schema and the support for this topic.
- **Enterprise Risk Management System (RMS) Integration:** While the platform's local Risk Service (PS-04) will handle high-speed, low-latency checks, certain complex risk calculations (e.g., portfolio-level concentration, value-at-risk) require a global view of the firm's positions. For these checks, the integration pattern will be a synchronous, request-response API call. The platform's Real-Time Risk Service will call the enterprise RMS API to get an approval decision before allowing an order to proceed. The performance of this synchronous call is critical. The Dependency Management Plan must therefore include a stringent SLA for the RMS API's latency and uptime, as any degradation in its performance will directly impact the trading platform's ability to execute trades.

Section 5: Non-Functional Requirements (NFRs) Alignment

The original project plan was notably deficient in its specification of Non-Functional Requirements (NFRs). The new strategic document imposes a set of rigorous, institutional-grade NFRs that must be formally documented and treated as primary project constraints. These requirements are not mere suggestions; they are hard mandates that will dictate major technology choices, architectural patterns, and engineering practices. A new section dedicated to NFRs must be added to the project plan, including the following table.

The single most impactful NFR is the requirement for end-to-end latency of less than 500 microseconds. This

requirement fundamentally challenges the suitability of the originally proposed generic enterprise technology stack (e.g., Java 8, Spring, ActiveMQ). A sub-millisecond target places the system in the domain of high-performance computing, where factors such as network hops between services, data serialization overhead (e.g., JSON versus a binary format like Protocol Buffers), and even programming language-specific behaviors like JVM garbage collection pauses become dominant concerns.

This NFR forces the project to adopt a "performance-first" engineering culture. Technology choices can no longer be based solely on developer productivity or existing team skills; they must be rigorously benchmarked and proven to meet the latency budget. Consequently, the project plan must be updated to include a new, dedicated workstream for **"Performance Engineering."** This workstream will be responsible for creating a dedicated performance testing environment, continuously profiling the system under load, and evaluating alternative technologies where necessary. For example, the latency-critical Order Execution Gateway might need to be implemented in a language like C++ to avoid JVM-related latency jitter and achieve the required determinism.

5.1 Table of Non-Functional Requirements

Category	Requirement	Metric	Rationale/Source
Latency	End-to-end latency for the critical trading path (market data ingress to order egress).	<500 microseconds (μ s) at the 99th percentile.	To support high-frequency and latency-sensitive strategies, as mandated by the new performance targets.
Throughput	The system's core messaging backbone must handle peak message loads without degradation.	Process a peak of 100,000 messages per second on the core Kafka event bus.	Derived from the mandate to handle a 10x increase in trading volume and data.
Scalability	All services must be designed to handle increased load by adding more instances.	All microservices must be stateless and horizontally scalable. The system must support a 10x increase in trading volume by adding container instances with zero downtime.	To ensure the platform can grow with the business and handle market volatility.
Availability	The core trading path must be highly resilient to component failure.	99.99% uptime for all critical path services (Data Gateway, Strategy Service, Risk Service, Order Gateway).	Required for an institutional-grade trading platform. This necessitates a multi-AZ (Availability Zone) cloud deployment strategy.
Security	All inter-service communication and external APIs must be	All internal and external API endpoints must be secured via	To protect firm and client data and prevent unauthorized access to

	secure. Sensitive data must be protected.	mutual TLS (mTLS). All secrets (API keys, passwords, certificates) must be stored in and retrieved from a secure vault solution (e.g., HashiCorp Vault).	the trading system.
Auditability	All trading-related actions must be captured in a compliant, immutable log.	100% of order lifecycle events must be captured and stored in a WORM-compliant data store with verifiable integrity.	To meet regulatory requirements such as MiFID II and provide a definitive system of record.

Section 6: Consolidated Change Log and Implementation Guide

This final section provides the master reference for executing all the proposed updates to the "Features, Phases & Integration Strategy - Algorithmic Trading System" document. It serves as the tactical translation of the preceding strategic analysis, removing ambiguity and ensuring a consistent and accurate update.

6.1 Purpose and Usage

The following table is the definitive checklist for modifying the project plan. Each row represents a single, atomic change required to bring the document into alignment with the new strategic directives. The Project Plan Section & Page column provides the exact location for the change, the Original Text column helps in precisely identifying the content to be altered, and the Revised / New Text column provides the exact wording to be used. The Rationale & Traceability column links each change back to a specific mandate, providing the necessary context for governance and change management.

6.2 Master Change Implementation Log

Change ID	Project Plan Section & Page	Change Type	Original Text (for MODIFY/REPLACE/DELETE)	Revised / New Text (for ADD/MODIFY/REPLACE)	Rationale & Traceability
CHG-ARC-001	2.1 System Architecture	REPLACE	"The system will be developed using a monolithic architecture for rapid deployment. The core components will be tightly	"The system will be architected as a set of independent, containerized microservices communicating asynchronously via an	To align with the firm's strategic mandate for a scalable and extensible platform as per 'Strategy Document', Section 3.1: Architectural

			coupled within a single application."	event-driven backbone (Kafka). This approach prioritizes scalability, resilience, and independent deployability of components."	Mandate.
CHG-TEC-001	2.2 Technology Stack	MODIFY	"The system will be built using Java 8, the Spring Framework, and ActiveMQ for messaging."	"The primary technology stack will be Java 17 and the Spring Framework. The messaging backbone will be Apache Kafka. For latency-critical components, alternative technologies such as C++ will be evaluated. All services will be containerized using Docker."	To modernize the stack, adopt Kafka for high-throughput messaging, and acknowledge the need for performance-oriented technology choices to meet new NFRs.
CHG-RSK-001	4.2 Risk Management	REPLACE	"Pre-trade checks will be limited to simple order size and price limits configured in a local properties file."	"A dedicated, real-time risk management microservice will be the central point for all pre-trade risk checks. It will validate all order requests against a centralized, database-	To implement the Enhanced Risk & Compliance Framework as mandated by 'Strategy Document', Section 4.2.

				driven limits system. This service will also provide a system-wide 'kill-switch' functionality."	
CHG-DAT-001	3.1 Data Sources	REPLACE	"The system will consume real-time market data for equities from Broker X's proprietary feed."	"The system will be capable of ingesting, normalizing, and processing data from multiple, diverse sources via a dedicated Market Data Gateway. Initial sources will include equity market data, cryptocurrency exchange feeds, and alternative data sets via REST APIs."	To support the expanded data strategy required for advanced ML models and multi-asset trading as per 'Strategy Document', Section 4.1.
CHG-STR-001	3.2 Strategy Logic	REPLACE	"The initial trading strategy will be a simple moving average crossover, implemented directly in the Java application core."	"The system will support the execution of complex, Python-based Machine Learning models via a dedicated ML Strategy Execution Service. This service will provide a secure, containerized environment	To shift from simple rules-based algorithms to a sophisticated ML-driven approach as mandated by 'Strategy Document', Section 4.3.

				for hosting models developed by the quantitative research team."	
CHG-PHA-001	5.1 Development Phases	REPLACE	"Phase 1 (MVP) will focus on deploying the moving average strategy in the monolithic application to achieve first trade as quickly as possible."	"Phase 1 (MVP) is focused on Architectural Validation and Risk Reduction. The primary deliverable is an end-to-end 'tracer bullet' implementation that validates the core microservices architecture and CI/CD pipeline, not initial profitability."	To realign the project phases to prioritize retiring technical risk in the new complex architecture, as per the 'de-risk the architecture' recommendation in the 'Strategy Document'.
CHG-INT-001	6.0 Integration	REPLACE	"The system is designed to be self-contained, with the only external integration being the FIX connection to Broker X."	"The system is a highly connected platform. It will integrate with multiple external brokers and data feeds via a pluggable connectivity layer. Internally, it will integrate with the enterprise Data Lake and the central Risk Management System (RMS)	To reflect the system's new role as a central component in the firm's enterprise architecture, as mandated by 'Strategy Document', Section 5.1.

				via well-defined APIs and event streams."	
CHG-NFR-001	2.3 Performance	ADD	(Section likely does not exist or is minimal)	"A new section, 'Non-Functional Requirements', will be added, specifying hard targets for latency ($< 500\mu s$), throughput (100k msgs/sec), and scalability (10x volume growth). Refer to Section 5.1 of this update document for the full table."	To formally document and enforce the stringent performance and scalability requirements that are central to the new strategic vision.